DA2009 - Group Project - IRCSL Ethical Web Scraping

DA 2009 - Data Collection Methods II
Group 3

Name	Student ID	Phase
Chandrasekaran Sivaneswaran	25ada147	Phase 3 - Brokers and Loss Adjusters
Zainul Abdeen Athif	25ada023	Phase 4 - Document Engine
V. K. Rajapakse	25ada139	Phase 1 - Static Pages
P. P. M. Subasinsani	25ada029	Phase 2 - Insurance Companies

Phase 4 - Document Engine: PDF Downloads, Text, Tables and OCR

Done by: Zainul Abdeen Athif (25ada023)
Website: https://ircsl.gov.lk (Acts, Regulations, Circulars, Guidelines and Media Room)

In this phase we list the PDF documents in five sections of the website, download a sample of them safely with **streaming**, and extract the metadata, text and tables with **PyPDF2** and **pdfplumber**. Some documents are scanned images, so we use **OCR (PyTesseract)** for those.

1. Setup

- `PyPDF2` reads PDFs, their metadata and basic text.
- `pdfplumber` reads text that keeps the page layout, and tables.
- `requests` and `BeautifulSoup` find and download the files, with the same polite rules as Phase 1.

In [1]:
```
import os
import time
import urllib.robotparser
from urllib.parse import urljoin

import requests
import pandas as pd
from bs4 import BeautifulSoup
from PyPDF2 import PdfReader
import pdfplumber

USER_AGENT = "DA2009-StudentProject/1.0 (BSc Applied Data Analytics coursework; academic use only)"
DELAY_SECONDS = 3
MAX_FILE_MB = 25           # we do not download files bigger than this
SAMPLE_PER_SECTION = 2     # number of documents to download from each section

BASE_URL = "https://ircsl.gov.lk"
PDF_DIR, TEXT_DIR, OUT_DIR = "data/pdfs", "data/text", "data/processed"
for folder in [PDF_DIR, TEXT_DIR, OUT_DIR]:
    os.makedirs(folder, exist_ok=True)

session = requests.Session()
session.headers.update({"User-Agent": USER_AGENT})

rp = urllib.robotparser.RobotFileParser()
rp.set_url(BASE_URL + "/robots.txt")
rp.read()
for u in [BASE_URL + "/insurance-legislation/circulars/", BASE_URL + "/bfd_download/issue-7-march-
2026/",
          BASE_URL + "/wp-content/uploads/2026/05/issue-7-march-2026.pdf"]:
    print("allowed:", rp.can_fetch(USER_AGENT, u), u)


def polite_get(url):
```

```
    if not rp.can_fetch(USER_AGENT, url):
        print("Not allowed by robots.txt:", url)
        return None
    response = session.get(url, timeout=30)
    time.sleep(DELAY_SECONDS)
    return response.text if response.status_code == 200 else None
```

```
allowed: True https://ircsl.gov.lk/insurance-legislation/circulars/
allowed: True https://ircsl.gov.lk/bfd_download/issue-7-march-2026/
allowed: True https://ircsl.gov.lk/wp-content/uploads/2026/05/issue-7-march-2026.pdf
```

2. Finding the documents

The homepage has **no links ending in `.pdf`**. The documents are on separate section pages (found with the Phase 1 site map), and their links look like `/bfd_download/issue-7-march-2026/`; the website then redirects to the real PDF file.

When we inspected a section page, each document looked like this (shortened):

```
<a class="bfd-download-meta" href="https://ircsl.gov.lk/bfd_download/circular-no-02-of-2026-.../">
   <p class="bfd-download-title">Circular No. 02 of 2026 (17th April 2026) – Amendment ...</p>
   <small>( 281.22 KB | pdf )</small>
</a>
```

So we find every document with the CSS selector `a.bfd-download-meta`, read the title, size and link, change the size to MB, and remove documents that are listed twice.

In [2]:
```
SECTIONS = {
    "Media Room": BASE_URL + "/media-room/",
    "Acts": BASE_URL + "/insurance-legislation/regulation-of-insurance-industry/",
    "Regulations": BASE_URL + "/insurance-legislation/test3/",
    "Circulars": BASE_URL + "/insurance-legislation/circulars/",
    "Guidelines": BASE_URL + "/instructions-and-guidelines/guidelines/",
}

catalogue_rows = []
for section, url in SECTIONS.items():
    soup = BeautifulSoup(polite_get(url), "html.parser")
    for link in soup.select("a.bfd-download-meta"):
        size_text = link.select_one("small").get_text(" ", strip=True).strip("( )").replace(" ",
"")
        number, unit = size_text.split("|")[0].split()
        catalogue_rows.append({
            "section": section,
            "title": link.select_one("p.bfd-download-title").get_text(" ", strip=True),
            "size_mb": round(float(number) / 1024 if unit == "KB" else float(number), 2),
            "download_url": urljoin(url, link["href"]),
        })

catalogue = pd.DataFrame(catalogue_rows)
print("Document links found:", len(catalogue))
catalogue = catalogue.drop_duplicates(subset=["section", "download_url"]).reset_index(drop=True)
print("After removing duplicates:", len(catalogue))
catalogue.groupby("section").agg(documents=("title", "count"), total_mb=("size_mb", "sum")).round(1)
```

```
Document links found: 115
After removing duplicates: 111
```

Out[2]:

section	documents	total_mb
Acts	5	0.7
Circulars	65	38.1
Guidelines	20	18.7
Media Room	14	31.2
Regulations	7	0.4

3. Downloading the PDFs with streaming

Downloading all the files would mean more than 100 requests and data we do not need, so we download only the **2 newest documents from each section** (10 files). This can be changed with `SAMPLE_PER_SECTION`.

With `requests.get(url).content` the whole file is loaded into memory. With `stream=True` the file arrives in 8 KB pieces that we write one by one. Our function also:

1. skips files that are already downloaded, and checks robots.txt
2. checks the status code (requests follows the redirect from `/bfd_download/` automatically), the `Content-Type` and the file size
3. writes to a temporary `.part` file and renames it only when the download is complete
4. checks that the file starts with `%PDF`, because every real PDF starts with these characters

In [3]:

```
def download_pdf_stream(url):
    file_name = url.rstrip("/").split("/")[-1][:120] + ".pdf"
    path = os.path.join(PDF_DIR, file_name)
    result = {"download_url": url, "file_name": file_name, "path": path,
              "redirects": "", "final_url": "", "content_type": "", "size_mb": None}

    if os.path.exists(path):
        result["status"] = "already downloaded"
        result["size_mb"] = round(os.path.getsize(path) / 1000000, 2)
        return result

    if not rp.can_fetch(USER_AGENT, url):
        result["status"] = "not allowed by robots.txt"
        return result

    temp_path = path + ".part"
    try:
        with session.get(url, stream=True, timeout=60) as response:
            result["redirects"] = ", ".join(str(r.status_code) for r in response.history)
            result["final_url"] = response.url
            result["content_type"] = response.headers.get("Content-Type", "")

            if response.status_code != 200:
                result["status"] = "HTTP error " + str(response.status_code)
                return result
            if "pdf" not in result["content_type"].lower():
                result["status"] = "not a PDF"
                return result
            if int(response.headers.get("Content-Length", 0)) > MAX_FILE_MB * 1000000:
                result["status"] = "file too large"
                return result

            downloaded = 0
            with open(temp_path, "wb") as f:
                for chunk in response.iter_content(chunk_size=8192):
                    f.write(chunk)
                    downloaded = downloaded + len(chunk)

        with open(temp_path, "rb") as f:
            first_bytes = f.read(4)
        if first_bytes != b"%PDF":
            os.remove(temp_path)
            result["status"] = "not a valid PDF"
            return result

        os.rename(temp_path, path)
        result["status"] = "downloaded"
        result["size_mb"] = round(downloaded / 1000000, 2)

    except requests.exceptions.RequestException as error:
        if os.path.exists(temp_path):
            os.remove(temp_path)
        result["status"] = "failed: " + str(error)
```

```
    time.sleep(DELAY_SECONDS)
    return result
```

The `redirects` column shows 302, which means the website sent us from the `/bfd_download/` link to the real file in `/wp-content/uploads/`.

In [4]:
```
sample = catalogue.groupby("section").head(SAMPLE_PER_SECTION).reset_index(drop=True)
results = [download_pdf_stream(url) for url in sample["download_url"]]

download_log = sample[["section", "title", "download_url"]].merge(pd.DataFrame(results),
on="download_url")
print(download_log["status"].value_counts().to_dict())
download_log[["section", "file_name", "redirects", "content_type", "size_mb", "status"]]
```

```
{'downloaded': 10}
```

Out[4]:

	section	file_name	redirects	content_type	size_mb	status
0	Media Room	issue-7-march-2026.pdf	302	application/pdf	3.60	downloaded
1	Media Room	issue-6-sepetember-2024.pdf	302	application/pdf	2.32	downloaded
2	Acts	regulation-of-insurance-industry-amendment-act...	302	application/pdf	0.08	downloaded
3	Acts	regulation-of-insurance-industry-amendment-act...	302	application/pdf	0.06	downloaded
4	Regulations	revision-to-annual-fee-of-insurance-company.pdf	302	application/pdf	0.04	downloaded
5	Regulations	amendments-to-the-insurance-regulatory-commiss...	302	application/pdf	0.02	downloaded
6	Circulars	circular-no-02-of-2026-17th-april-2026-amendme...	302	application/pdf	0.29	downloaded
7	Circulars	circular-no-01-of-2026-3rd-march-2026-enforcem...	302	application/pdf	1.14	downloaded
8	Guidelines	01-09-2026-guidelines-on-universal-life-credit...	302	application/pdf	4.13	downloaded
9	Guidelines	01-09-2026-guidelines-on-management-of-partici...	302	application/pdf	4.16	downloaded

4. Checking the PDFs and reading the metadata (PyPDF2)

For each file we check that it opens without an error, whether it is encrypted and how many pages it has. We also read the metadata saved inside the file, such as the program that created it.

In [5]:
```
def check_pdf(path):
    try:
        reader = PdfReader(path)
        meta = reader.metadata or {}
        return {"valid_pdf": True, "encrypted": reader.is_encrypted, "pages": len(reader.pages),
                "meta_title": meta.get("/Title"), "meta_creator": meta.get("/Creator"),
                "created": meta.get("/CreationDate")}
    except Exception as error:
        return {"valid_pdf": False, "error": str(error)}


documents = download_log[download_log["status"].isin(["downloaded", "already
downloaded"])].reset_index(drop=True)
documents = pd.concat([documents, pd.DataFrame([check_pdf(p) for p in documents["path"]])], axis=1)
documents[["section", "file_name", "valid_pdf", "encrypted", "pages", "meta_creator", "created"]]
```

Out[5]:

	section	file_name	valid_pdf	encrypted	pages	meta_creator	created
0	Media Room	issue-7-march-2026.pdf	True	False	25	Adobe InDesign 21.3 (Windows)	D:20260429103229+05'30'
1	Media Room	issue-6-sepetember-2024.pdf	True	False	4	Adobe InDesign 19.5 (Windows)	D:20240925093024+05'30'
2	Acts	regulation-of-insurance-industry-amendment-act...	True	False	9	PageMaker 7.0	D:20171031123307+05'30'
3	Acts	regulation-of-insurance-industry-amendment-act...	True	False	29	PageMaker 7.0	D:191110228160411
4	Regulations	revision-to-annual-fee-of-insurance-company.pdf	True	False	2	PScript5.dll Version 5.2.2	D:20250115133201+05'30'
5	Regulations	amendments-to-the-insurance-regulatory-commiss...	True	False	1	PageMaker 7.0	D:20190101010055+05'30'
6	Circulars	circular-no-02-of-2026-17th-april-2026-amendme...	True	False	1	NaN	NaN
7	Circulars	circular-no-01-of-2026-3rd-march-2026-enforcem...	True	False	3	NaN	NaN
8	Guidelines	01-09-2026-guidelines-on-universal-life-credit...	True	False	7	Microsoft® Word 2016	D:20260819112039+05'30'
9	Guidelines	01-09-2026-guidelines-on-management-of-partici...	True	False	10	Microsoft® Word 2016	D:20260819111816+05'30'

The circulars have no metadata at all. This was our first sign that they are scanned documents.

5. Extracting the text

5.1 PyPDF2 and pdfplumber on the same document

PyPDF2 gives basic text and often joins lines together. pdfplumber with `layout=True` keeps the layout of the page, such as spacing and columns.

In [6]:

```
def extract_pypdf2(pdf_path):
    reader = PdfReader(pdf_path)
    all_text = []
    for page_number, page in enumerate(reader.pages, start=1):
        try:
            text = page.extract_text()
            if text:
                all_text.append(f"\n--- PAGE {page_number} ---\n")
                all_text.append(text)
        except Exception as e:
            print(f"PyPDF2 error on page {page_number}: {e}")
    return "\n".join(all_text)


def extract_pdfplumber(pdf_path):
    all_text = []
    with pdfplumber.open(pdf_path) as pdf:
        for page_number, page in enumerate(pdf.pages, start=1):
            try:
                text = page.extract_text(layout=True)
                if text:
                    all_text.append(f"\n--- PAGE {page_number} ---\n")
                    all_text.append(text)
            except Exception as e:
                print(f"pdfplumber error on page {page_number}: {e}")
    return "\n".join(all_text)


example = documents[documents["section"] == "Acts"].iloc[0]["path"]
text_a = extract_pypdf2(example)
text_b = extract_pdfplumber(example)
print("Document:", example)
print("PyPDF2 characters:", len(text_a), "| pdfplumber characters:", len(text_b), "(includes the
layout spaces)")
print("\n===== PyPDF2 =====\n", text_a[:500])
print("\n===== pdfplumber (layout=True) =====\n", text_b[:700])
```

```
Document: data/pdfs/regulation-of-insurance-industry-amendment-act-no-23-of-2017.pdf
PyPDF2 characters: 10473 | pdfplumber characters: 48281 (includes the layout spaces)

===== PyPDF2 =====

--- PAGE 1 ---

PARLIAMENT OF THE DEMOCRATIC
SOCIALIST REPUBLIC OF
SRI LANKA
REGULATION  OF  INSURANCE  INDUSTRY
(AMENDMENT)  ACT,  No.  23  OF  2017
Printed on the Order of Government[Certified on 19th of October, 2017]
PRINTED  AT THE DEPARTMENT  OF GOVERNMENT  PRINTING , SRI LANKA
TO BE PURCHASED  AT THE GOVERNMENT  PUBLICATIONS  BUREAU , COLOMBO  5
Price : Rs. 6.00 Postage : Rs. 10.00
Published as a Supplement to Part II of the Gazette of the Democratic
Socialist Republic of Sri Lanka of Oc

===== pdfplumber (layout=True) =====

--- PAGE 1 ---
```

5.2 What type of PDF is each file?

In the lecture we learned to find the PDF type before choosing a tool. We count the characters and the pages with images:

- **Text-based**: has text and few images -> PyPDF2 or pdfplumber
- **Hybrid**: text and images on most pages -> pdfplumber
- **Image-based (scanned)**: almost no text and an image on every page -> OCR

Then we extract the plain text of every file with pdfplumber.

In [7]:
```python
def pdf_type(path):
    with pdfplumber.open(path) as pdf:
        pages = len(pdf.pages)
        chars_per_page = sum(len(page.chars) for page in pdf.pages) / pages
        pages_with_images = sum(1 for page in pdf.pages if page.images)
        text = "\n".join((page.extract_text() or "") for page in pdf.pages)
    if chars_per_page < 50 and pages_with_images == pages:
        kind = "image-based (scanned)"
    elif pages_with_images >= pages / 2:
        kind = "hybrid"
    else:
        kind = "text-based"
    return pd.Series({"chars_per_page": round(chars_per_page), "pages_with_images": pages_with_images,
                      "pdf_type": kind, "text": text})


documents[["chars_per_page", "pages_with_images", "pdf_type", "text"]] =
documents["path"].apply(pdf_type)
documents["text_source"] = "pdfplumber"
documents[["section", "file_name", "pages", "chars_per_page", "pages_with_images", "pdf_type"]]
```

Out[7]:

	section	file_name	pages	chars_per_page	pages_with_images	pdf_type
0	Media Room	issue-7-march-2026.pdf	25	3638	25	hybrid
1	Media Room	issue-6-sepetember-2024.pdf	4	4006	2	hybrid
2	Acts	regulation-of-insurance-industry-amendment-act...	9	1124	1	text-based
3	Acts	regulation-of-insurance-industry-amendment-act...	29	1510	1	text-based
4	Regulations	revision-to-annual-fee-of-insurance-company.pdf	2	1101	1	hybrid
5	Regulations	amendments-to-the-insurance-regulatory-commiss...	1	2343	0	text-based
6	Circulars	circular-no-02-of-2026-17th-april-2026-amendme...	1	0	1	image-based (scanned)
7	Circulars	circular-no-01-of-2026-3rd-march-2026-enforcem...	3	0	3	image-based (scanned)
8	Guidelines	01-09-2026-guidelines-on-universal-life-credit...	7	2697	0	text-based
9	Guidelines	01-09-2026-guidelines-on-management-of-partici...	10	2714	0	text-based

6. OCR for the scanned circulars

OCR (Optical Character Recognition) reads text from an image. Our steps:

1. turn each PDF page into an image with pdfplumber (`page.to_image(resolution=300)`); a higher resolution gives better results
2. change the image to grayscale with PIL (`.convert("L")`)
3. read the text with `pytesseract.image_to_string()`

Tesseract is the OCR program and PyTesseract connects it to Python. Only the image-based files go through OCR.

In [8]:

```
import pytesseract


def ocr_pdf(path):
    pages_text = []
    with pdfplumber.open(path) as pdf:
        for page in pdf.pages:
            image = page.to_image(resolution=300).original.convert("L")   # grayscale with PIL
            pages_text.append(pytesseract.image_to_string(image))
    return "\n".join(pages_text)


scanned = documents["pdf_type"] == "image-based (scanned)"
print("Scanned documents:", scanned.sum())
for i in documents[scanned].index:
    documents.at[i, "text"] = ocr_pdf(documents.at[i, "path"])
    documents.at[i, "text_source"] = "OCR (pytesseract)"
print(documents.loc[scanned, "text"].iloc[0][:1000])
```

```
Scanned documents: 2
& ©omD) Oedn BHOn eemBees sen
© QOHind STUBS QYHGYPMBULGSSHO ApenoIsG Up
INSURANCE REGULATORY COMMISSION OF SRI LANKA

INSURANCE I] 6 One, arcmnsic aes Il wimg, slypse Carygup Level 11, East Tower

pores OF Sela GBEC Dasdom DH Hil AMLOWUIL World Trade Centre

SRI LANKA cane® O1. G Gomd. Gasmapibty 01, Qevtuonss. Colombo 01, Sri Lanka.
( +9411 2396184-9 Gq! +9411 2396190 ® wwwairest.goulk

REGULATION OF INSURANCE INDUSTRY ACT — CIRCULAR

17" April 2026 No. 02 of 2026

ee
Amendment to the Circular # 04 of 2025 issued on 08 December 2025

on the Requirements on Premium Payment for Policies of General Insurance

In view of the Direction # 5 of 2026 issued on 31% March 2026, Circular # 04 of 2025 shall be amended
as follows:

A General Insurance Company may grant a credit period to the insured of a Motor Insurance Policy to
pay the full insurance premium to the Insurance Company. However, such credit period shall not exceed
30 days from the inception of cover.

This Circular shall be effect
```

7. Tables inside the PDFs (pdfplumber)

pdfplumber finds tables by looking at lines and the position of the text. `extract_tables()` gives each table as a list of rows, which we change into a DataFrame with the first row as column names.

In [9]:
```
all_tables = []
documents["tables_found"] = 0
for i in documents.index:
    with pdfplumber.open(documents.at[i, "path"]) as pdf:
        for page_number, page in enumerate(pdf.pages, start=1):
            for table in page.extract_tables():
                if table and len(table) > 1:
                    df = pd.DataFrame(table[1:], columns=table[0])
                    df.insert(0, "page", page_number)
                    df.insert(0, "file_name", documents.at[i, "file_name"])
                    all_tables.append(df)
                    documents.at[i, "tables_found"] += 1

print("Tables found:", len(all_tables))
display(documents[["section", "file_name", "pdf_type", "tables_found"]])

# example: the first table from the guidelines (a blank form insurers must fill in)
guideline_tables = [t for t in all_tables if "guidelines" in t["file_name"].iloc[0]]
guideline_tables[0].head()
```

```
Tables found: 27
```

	section	file_name	pdf_type	tables_found
0	Media Room	issue-7-march-2026.pdf	hybrid	6
1	Media Room	issue-6-sepetember-2024.pdf	hybrid	15
2	Acts	regulation-of-insurance-industry-amendment-act...	text-based	0
3	Acts	regulation-of-insurance-industry-amendment-act...	text-based	0
4	Regulations	revision-to-annual-fee-of-insurance-company.pdf	hybrid	0
5	Regulations	amendments-to-the-insurance-regulatory-commiss...	text-based	1
6	Circulars	circular-no-02-of-2026-17th-april-2026-amendme...	image-based (scanned)	0
7	Circulars	circular-no-01-of-2026-3rd-march-2026-enforcem...	image-based (scanned)	0
8	Guidelines	01-09-2026-guidelines-on-universal-life-credit...	text-based	3
9	Guidelines	01-09-2026-guidelines-on-management-of-partici...	text-based	2

Out[9]:

	file_name	page	Particular	Amount (LKR)
0	01-09-2026-guidelines-on-universal-life-credit...	6	Annual premium amount	
1	01-09-2026-guidelines-on-universal-life-credit...	6	Basic Sum Assured1	
2	01-09-2026-guidelines-on-universal-life-credit...	6	Unit fund value at the start of the period2	
3	01-09-2026-guidelines-on-universal-life-credit...	6	Premiums paid during the period	
4	01-09-2026-guidelines-on-universal-life-credit...	6	Premiums allocated during the period	

8. Saving the results

We save one row for each document, the full text of each document in a `.txt` file (one Excel cell can only hold 32,767 characters, and one newsletter has about 90,000), and every table in its own Excel sheet.

In [10]:

```
documents["word_count"] = documents["text"].str.split().str.len()
documents["text_file"] = TEXT_DIR + "/" + documents["file_name"].str.replace(".pdf", ".txt")
for path, text in zip(documents["text_file"], documents["text"]):
    with open(path, "w", encoding="utf-8") as f:
        f.write(text)

catalogue.to_csv(OUT_DIR + "/document_catalogue.csv", index=False)
document_summary = documents.drop(columns=["text"])
document_summary.to_csv(OUT_DIR + "/document_summary.csv", index=False)
document_summary.to_json(OUT_DIR + "/document_summary.json", orient="records", indent=2)
with pd.ExcelWriter(OUT_DIR + "/pdf_tables.xlsx") as writer:
    for n, table in enumerate(all_tables, start=1):
        table.to_excel(writer, sheet_name="table_" + str(n), index=False)

print("Documents in the list:", len(catalogue), "| PDFs downloaded:", len(documents), "| tables:",
len(all_tables))
document_summary[["section", "title", "pages", "pdf_type", "text_source", "word_count",
"tables_found"]]
```

```
Documents in the list: 111 | PDFs downloaded: 10 | tables: 27
```

Out[10]:

	section	title	pages	pdf_type	text_source	word_count	tables_found
0	Media Room	Issue 7 – March 2026	25	hybrid	pdfplumber	12024	6
1	Media Room	Issue 6 – September 2024	4	hybrid	pdfplumber	2248	15
2	Acts	Regulation of Insurance Industry (Amendment) A...	9	text-based	pdfplumber	1709	0
3	Acts	Regulation of Insurance Industry (Amendment) A...	29	text-based	pdfplumber	7582	0
4	Regulations	Revision to annual fee of Insurance Company	2	hybrid	pdfplumber	363	0
5	Regulations	Amendments to the Insurance Regulatory Commiss...	1	text-based	pdfplumber	346	1
6	Circulars	Circular No. 02 of 2026 (17th April 2026) – Am...	1	image-based (scanned)	OCR (pytesseract)	208	0
7	Circulars	Circular No.01 of 2026 (3rd March 2026) – Enfo...	3	image-based (scanned)	OCR (pytesseract)	1096	0
8	Guidelines	01.09.2026 – Guidelines on Universal Life Cred...	7	text-based	pdfplumber	2734	3
9	Guidelines	01.09.2026 – Guidelines on Management of Parti...	10	text-based	pdfplumber	3913	2

9. Ethical considerations and limitations

- We checked robots.txt for the pages, the download links and the uploads folder, and again before every download.
- We waited 3 seconds after every page and every file, with our student User-Agent.
- We listed all the documents (5 page requests) but downloaded only 10. Files already downloaded are not downloaded again, and the 25 MB limit and streaming keep memory use low.

- All the documents are public documents published by IRCSL. We did not log in or bypass anything.
- **Limitations:** we only downloaded a sample (`SAMPLE_PER_SECTION` can be increased). OCR can make mistakes. The Sinhala text in older gazettes uses an old font, and the tables in the guidelines are blank forms.